CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

TECHNICAL ASSESSMENT REPORT

NETVISTA 3D — BIT STUFFING & DESTUFFING IN HDLC PROTOCOL

Individual Performance-Based Assessment Package

Course Code & Name:	ITUC201 — Fundamentals of Computer Networks
Assigned Topic:	Bit Stuffing in HDLC Protocol
Student ID / Enrollment:	25CS016
Student Name:	Bhavy Gol
Class / Section / Group:	B.Tech CE / Semester 3
Academic Year:	2026–2027 (Semester 3)
Submission Deadline:	22 October 2026

1. Abstract

This technical report presents a complete educational implementation and empirical performance evaluation of sender bit stuffing and receiver destuffing in High-Level Data Link Control (HDLC) framing. Developed under Student ID 25CS016 for ITUC201 (Fundamentals of Computer Networks) at CHARUSAT, the project includes a production-grade Python core library, an interactive Tkinter graphical visualizer, a command-line demonstrator, and an automated benchmark test suite. All mandatory test cases involving runs of five consecutive 1-bits were validated with 100% roundtrip restoration accuracy.

2. Objectives

The primary academic and technical objectives of this project are:
1. Implement bit-level sender stuffing to prevent false flag delimiter occurrence in payload data.
2. Implement receiver destuffing to accurately recover original binary payloads and detect malformed bit streams.
3. Build an educational framing visualizer incorporating HDLC flag delimiters (01111110).
4. Perform empirical benchmarking across varying 1-bit density patterns to analyze bit stuffing overhead.
5. Deliver fully reproducible executable source code, automated unit tests, and solved university-style examination problems.

3. Introduction to HDLC Protocol

High-Level Data Link Control (HDLC) is a bit-oriented Data Link Layer protocol standardized by ISO (ISO/IEC 13239). It governs synchronous serial data transmission across point-to-point and multipoint links. Unlike character-oriented protocols, HDLC processes arbitrary bit streams without relying on ASCII/character encodings. Data frames are demarcated by a unique 8-bit flag sequence: 01111110 (0x7E).

4. Theory of Bit Stuffing and Destuffing

To maintain bit-oriented data transparency, arbitrary data payloads must be transmitted without accidentally containing the flag sequence 01111110.
• **Sender Rule:** After every sequence of five consecutive 1-bits ('11111') in the payload, the sender automatically inserts one '0' bit. This ensures that a sequence of six consecutive 1-bits can NEVER occur inside the stuffed payload data region.
• **Receiver Rule:** The receiver continuously monitors the incoming bit stream. Whenever five consecutive 1-bits are detected in the stuffed payload region, the receiver examines the 6th bit. If the 6th bit is '0', it is identified as a stuffed zero bit and discarded, restoring the original payload.
• **Delimiter Protection:** Opening and closing flag delimiters (01111110) are added AFTER stuffing and removed BEFORE destuffing.

5. Problem Statement & Data Transparency Challenge

Consider a payload containing user data that matches the HDLC flag sequence '01111110'. Without bit stuffing, transmitting this raw payload wrapped in flags produces:

[01111110] 01111110 [01111110]

The receiver's physical layer hardware detects the middle 01111110 sequence as an early closing flag, causing premature frame termination and total loss of subsequent data. Bit stuffing converts the payload '01111110' into '011111010', breaking the 6-ones pattern while allowing the receiver to perfectly restore '01111110'.

6. Project Tools and Environment

The project was executed in a clean Python 3 environment:
• Core Tool: Python 3.9+
• GUI Framework: Python Tkinter (standard library)
• Testing Framework: Python unittest (standard library)
• Visualization & Graphics: Matplotlib 3.9+, Pillow
• Document Automation: python-docx, reportlab, python-pptx

7. Implementation Methodology

The implementation follows a modular software engineering architecture:
1. Functional Core (src/hdlc_core.py): Pure algorithms for stuffing, destuffing, and framing.
2. CLI Application (src/hdlc_cli.py): Console-based demonstration and interactive audit mode.
3. Graphical Dashboard (src/hdlc_gui.py): Desktop visualizer with real-time stream highlights.
4. Experiment Runner (src/experiment_runner.py): Benchmark suite producing CSV/JSON evidence.
5. Test Suite (tests/test_hdlc.py): Automated test cases covering normal and boundary cases.

8. Sender Bit Stuffing Algorithm & Flowchart

The sender algorithm operates in O(N) time complexity relative to input bit length N:
Count consecutive 1-bits. After every 5 consecutive 1s, append an inserted 0 bit.

9. Receiver Bit Destuffing Algorithm & Error Validation

The receiver destuffing algorithm removes inserted zeros and validates stream integrity:
Inspect bits after 5 consecutive 1s. Discard inserted 0 bit. Raise exception on malformed stream.

10. Architecture and Modular Design

The codebase is structured cleanly into decoupled packages under NetVista3D_HDLC_Bit_Stuffing_25CS016.

11. HDLC-Style Flagged Frame Demonstration

An educational HDLC frame is represented as:
FLAG (01111110) | STUFFED PAYLOAD | FLAG (01111110)
For input payload '01111110', the stuffed body is '011111010'. Complete framed stream: 01111110 011111010 01111110.

12. Python Implementation Explanation

The core functions in `hdlc_core.py` enforce strict binary string validation (`validate_binary_string`).

13. Experiment Design & Benchmarking Methodology

The automated benchmark runner (`experiment_runner.py`) evaluates 25 total test cases including mandatory and synthetic patterns.

14. Test Inputs and Actual Empirical Results

Summary of mandatory test cases:
1. Case 1 (101010): Stuffed = 101010 | Overhead = 0.00% | PASS
2. Case 2 (11111): Stuffed = 111110 | Overhead = 20.00% | PASS
3. Case 3 (111111): Stuffed = 1111101 | Overhead = 16.67% | PASS
4. Case 4 (01111110): Stuffed = 011111010 | Overhead = 12.50% | PASS
5. Case 5 (11111011111): Stuffed = 1111100111110 | Overhead = 18.18% | PASS

15. Bit Overhead Calculations and Quantitative Analysis

Stuffing Overhead (%) = (Inserted Zero Bits / Original Payload Length) * 100%. Dynamic overhead ranges between 0.00% and 20.00%.

16. Screenshots & Visual Evidence

Generated assets located in evidence/charts/ and evidence/screenshots/.

17. Analysis and Technical Interpretation

Experimental data matches theoretical bit stuffing metrics.

18. Fully Worked University-Style Examination Question & Solution

QUESTION: Given input bit stream '011111101111101', show sender bit stuffing, construct HDLC frame, show receiver destuffing, and calculate overhead percentage.

SOLUTION:
1. Original Payload: 011111101111101 (15 bits)
2. Stuffed Payload: 01111101011111001 (17 bits, 2 zeros inserted)
3. HDLC Frame: 01111110 | 01111101011111001 | 01111110 (33 total bits)
4. Overhead: (2 / 15) * 100% = 13.33%

19. Limitations and Protocol Scope Differences

This educational implementation focuses specifically on Data Link Layer payload bit stuffing and flag framing. Full production HDLC includes Address, Control, and FCS fields.

20. Conclusion

The project successfully implements and validates HDLC bit stuffing with 100% test pass rate.

21. References

1. Tanenbaum, A. S. (2011). Computer Networks (5th ed.).
2. Forouzan, B. A. (2013). Data Communications and Networking (5th ed.).
3. ISO/IEC 13239:2002 HDLC Procedures.

22. Reproducibility Instructions

1. Run unit tests: python3 -m unittest discover -s tests -p 'test_*.py'
2. Run experiment runner: python3 src/experiment_runner.py
3. Launch GUI: python3 src/hdlc_gui.py

23. AI Assistance Declaration

AI assistance was used in accordance with CHARUSAT policy for formatting and layout. All logic was validated under Student ID 25CS016.